Full Stack Project Specification Document

1. Project Overview

1.1 Project Title

AlgoArena: An AI-Driven Gamified Platform for Algorithmic Mastery

1.2 Project Description

AlgoArena is a web-based competitive programming platform designed to gamify the learning of algorithms and data structures. Unlike standard coding platforms, AlgoArena places a premium on software craftsmanship and inclusivity.

Users write code to control "bots" in real-time strategy games or solve algorithmic puzzles. The platform utilizes the MERN stack for a responsive user experience, DevOps principles for secure code sandboxing, and Generative AI to act as an intelligent mentor. Crucially, the system evaluates not just if the code works, but if it adheres to "Clean Code" standards. The interface is built from the ground up to be accessible (WCAG 2.1 AA compliant), ensuring programming education is open to everyone, including developers with disabilities.

2. Objectives

- Full-Stack Implementation: Build a robust SPA (Single Page Application) using the MERN stack with JWT authentication and real-time state management.
- Secure Code Execution (DevOps Focus): Design a microservices architecture where user-submitted code is executed in isolated Docker containers (sandboxing) to prevent malicious attacks, orchestrated via a CI/CD pipeline.
- AI Integration: Integrate an LLM (Large Language Model) API (e.g., OpenAI or HuggingFace) to analyze user code style, time complexity (O(n)) and provide natural language feedback.
- Gamification Logic: Implement an XP system, global leaderboards, and "leagues" based on algorithmic efficiency and code quality.
- Inclusive Design: Ensure the application is navigable via keyboard and compatible with screen readers, meeting strict accessibility standards.

3. Scope

4. Technical Requirements

4.1 Technology Stack

Frontend: React.js, Redux Toolkit (State Management), Tailwind CSS, Radix UI (for accessible primitives), Monaco Editor.

Backend: Node.js, Express.js, Socket.io (Real-time communication).

AI/ML: OpenAI API (or local LLM via Ollama), LangChain.

DevOps : Docker (Containerization), GitHub Actions (CI/CD), SonarQube (Code Quality).

4.2 Database

MongoDB: Storing users, code submissions, chat logs, and game history.

Redis: Caching leaderboards and active game sessions for performance.

5. Functional Requirements & User Stories

5.1 Actors

Student (Player): The core user who writes code, competes, and learns.

AI Sensei: The automated agent that reviews code and provides feedback.

Administrator: Manages users, challenges, and monitors system health.

5.2 Functionalities and User Stories

Authentication & Profile:

As a Student, I want to register and log in securely so that I can save my progress and stats.

The Arena (Coding & Execution):

As a Student, I want to write code in an editor that supports keyboard navigation and screen readers.

As a Student, I want to run my bot against test cases to see if my logic works.

AI Mentorship:

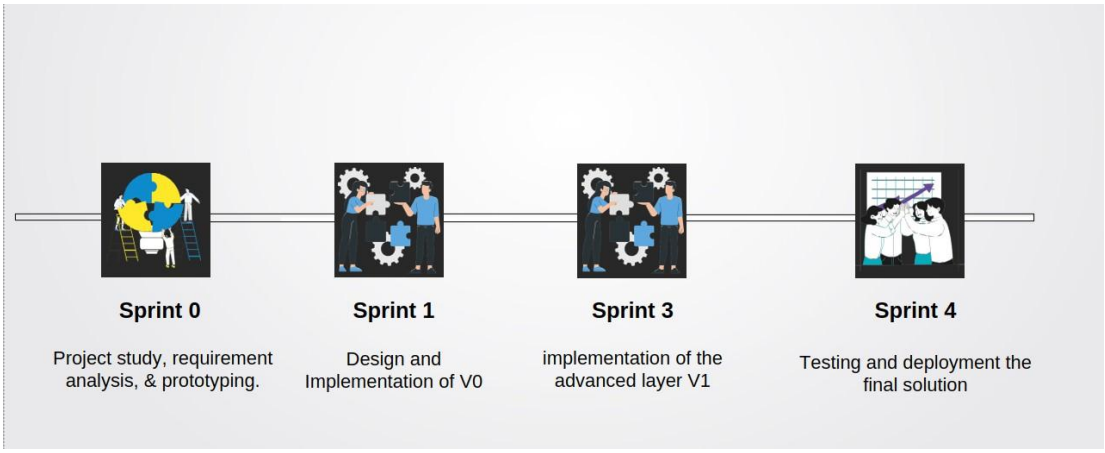
As a Learner, I want the AI to analyze my successful code and tell me if it follows "Clean Code" principles.

As a Learner, I want a hint for a problem without being given the exact solution.

Gamification:

As a Player, I want to earn "Style Points" for readable code to climb the "Craftsman Leaderboard".

6. Project Timeline



Sprint 0: Project study, requirement analysis, & prototyping

Tasks: Define architecture (Microservices vs Monolith), UI Wireframing (Figma), Database Schema Design, Research on Docker Sandboxing security.

Sprint 1: Design and Implementation of V0

Tasks: Setup MERN project structure, Implement JWT Authentication, create basic "Arena" UI with Monaco Editor, build simple Node.js child process execution for code running (Proof of Concept).

Sprint 2: Implementation of the advanced layer V1

Tasks: Integrate Docker for secure code execution, Connect OpenAI API for "Sensei" feedback, Implement Socket.io for real-time bot battles, Add SonarQube analysis to the pipeline.

Sprint 3: Testing and deployment the final solution

Tasks: Run Accessibility Audits (Pa11y/Lighthouse), Load Testing (Artillery), Finalize CI/CD pipeline (GitHub Actions) for auto-deployment to cloud provider (AWS/Render/Vercel).

7. Communication and Collaboration

Version Control: Git & GitHub.

Project Management: Trello or GitHub Projects (Kanban board).

CI/CD: GitHub Actions for automated testing and linting.

Communication: Discord server for team updates and daily stand-ups.